



pythonTM

Programmation python

DI & IG & RT

MA Amar

medabdellahiamar@yahoo.fr

AU 2023-2024



pythonTM

Chapitre 2 Instructions Conditionnelles et itératives



Instructions Conditionnelles

[elif ...] else ...]

Qu'est-ce qu'une instruction conditionnelle ?

Définition : Une instruction conditionnelle est une instruction qui dépend d'une condition. Certaines parties du code sont exécutées ou ignorées selon que la condition est vérifiée ou non.

Syntaxe

```
if conditions :  
    Bloc d'instructions1  
else :  
    Bloc d'instructions2
```

Exécuter un premier bloc si une condition est vraie, un deuxième sinon (le bloc else)

if (<condition>) : # en-tête
si la condition est vraie
Instruction 1>
Instruction n>



Même niveaux d'indentation

Remarques

- Les parenthèses ne sont pas obligatoires
- L'indentation est obligatoire pour les instructions de **if**
- Pour les conditions, on utilise les opérateurs de comparaison

Opérateurs de comparaison

- **==** : pour tester l'égalité
- **!=** : pour tester l'inégalité
- **>** : supérieur à
- **<** : inférieur à
- **>=** : supérieur ou égal à
- **<=** : inférieur ou égal à



Instructions Conditionnelle

Exemple

Quand on calcule le reste dans la division par 2 d'un entier, il n'y a que deux cas : si le reste est 0, cet entier est pair et, sinon, il est impair. On traduit cette alternative par une instruction conditionnelle:

```
n= int(input("donner un entier:"))  
if n %2==0:  
    print(n, "est un nombre pair ")  
else :  
    print(n, "est un nombre impair")
```

Avertissement

N'oubliez pas les double-points, et faites attention à l'indentation (en pratique de 4 caractères)



Instructions Conditionnelle

Exemple

Pour tester si un nombre donné par l'utilisateur est positif (y compris 0) ou négatif:

```
x= float(input("donner un nombre:"))
```

```
if (x >= 0):
```

```
    print(x, " est positif")
```

```
else:
```

```
    print(x, " est négatif")
```





Instructions Conditionnelle

Exemple

```
note= float(input( "donner votre Note sur 20 : "))  
if note>=10.0:  
    print("J'ai la moyenne")  
else:  
    print("C'est en dessous de la moyenne")  
print("Fin du programme")
```



Instructions Conditionnelles

Lorsqu'il n'y a pas d'instructions à exécuter dans le cas où la condition est fausse, on écrit juste :

if (condition):

Bloc d'instructions

Exemple:

```
x= float(input("donner un nombre: "))
```

```
if (x > 0):
```

```
    print( x , " est positif " )
```




Instructions Conditionnelles

if imbriquées

```
if condition1:  
    instructions 1  
else:  
    if conditions2 :  
        instructions 2  
    else:  
        if condition3 :  
            instructions 3  
        else:  
            instructions4
```

On peut enchaîner les conditions avec **elif**

```
if condition1:  
    instructions 1  
elif conditions2 :  
    instructions 2  
elif condition3 :  
    instructions 3  
else:  
    instructions4
```



Instructions Conditionnelles

if imbriquées (**Exemple**)

Ecrire un programme qui permet de tester si un nombre donné par l'utilisateur est positif, négatif ou nul:

```
x=int(input("Donner un nombre"))
if (x > 0):
    print(x, " est positif")
elif (x == 0):
    print(x, " est nul")
else:
    print(x, " est négatif")
```



Instructions Conditionnelles

Opérateurs logiques

- and : et
- or : ou
- not : non



Instructions Conditionnelles

Tester plusieurs conditions (en utilisant des opérateurs logiques)

```
if (condition1 and condition2 or condition3):  
    # instructions  
[else ...]
```



Instructions Conditionnelles

Exercice

Écrire un script **Python** qui

- demande à l'utilisateur de saisir deux valeurs numériques
- affiche le signe du résultat de la multiplication sans calculer le produit



Instructions Conditionnelles

Qu'affiche exactement ce programme après exécution des instructions suivantes?

a=14

b=5

if a>=10:

c=a//b

print(c)

else:

c=a%b

print(c)

print(c)





Instructions Conditionnelles

Qu'affiche exactement ce programme après exécution des instructions suivantes?

a=4

b=7

if a>=10:

print(a+b)

else:

print(b*a)





Instructions Conditionnelles

Qu'affiche exactement ce programme après exécution des instructions suivantes?

a=4

b=7

print(a,b)

if a>=10:

a=a+5

b=b*a

print(a,b)

else:

print(a,b)

print(" le bloc de ELSE")

print(" fin")





EXERCICES

1. Écrire un programme max.py, qui demande de saisir 2 valeurs et qui affiche la plus grande des 2 valeurs.





EXERCICES

2. Écrire un programme `min_max.py`, qui demande de saisir 2 valeurs et qui affiche la plus petite et la plus grande des 2 valeurs.
3. Écrire un programme `max3.py`, qui demande de saisir 3 valeurs et qui affiche la valeur la plus grande.



EXERCICES

Ecrire un programme qui lit le potentiel hydrogène Ph d'une solution puis affiche sa nature selon les cas suivants :

Ph inférieur à 7 est **acide**

Ph supérieur à 7 est **basique**

Ph égal de 7 est **neutre**



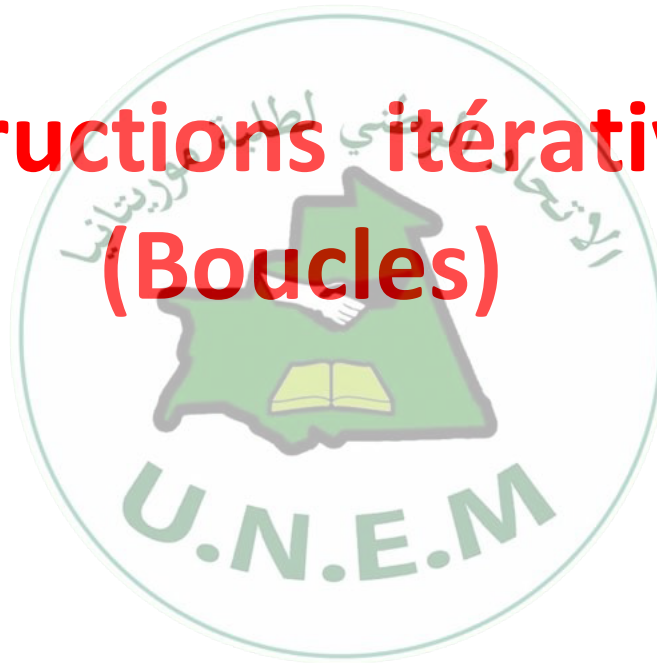
Instruction **pass**

- Si, pour certaines conditions, on n'a rien à faire, on peut utiliser **pass**

```
x = 0
if (x > 0):
    print (x, " est positif")
elif (x == 0):
    pass
else:
    print (x, " est négatif")
```



Instructions itératives (Boucles)



Les structures répétitives



- Une structure répétitive existe souvent dans un **programme** qu'une même action soit répétée plusieurs fois.
 - **Ecrire un programme qui permet d'afficher le mot "bonjour" 50 fois.**
 - 50 : nombre de répétition (itération).
 - Le traitement à répéter le mot "Bonjour".
- Pour gérer ces cas, on utilise des boucles qui ont pour effet de répéter plusieurs fois une même instruction.
- **Deux formes existent** : la première, si le nombre de répétitions est connu avant l'exécution, la seconde s'il n'est pas connu.



Les structures répétitives

Structures itératives

- while : pour un nombre d'itérations connu ou inconnus
- for : pour un nombre d'itérations connu



structure répétitive for

La structure répétitive **for** permet de répéter une liste d'instructions un nombre connu de fois.

Boucle **for**

```
for elt in iterable:  
    # instruction[s]
```

Et si on a besoin d'afficher ou de manipuler des valeurs (de 0 à n par exemple) on utilise la fonction **range**

Pour manipuler des valeurs (de 0 a 4 **par exemple**) on utilise la fonction **range(5)**



Exemple

```
for i in range (5) :  
    print(i)
```

Le résultat est

0
1
2
3
4



Exemple

On peut aussi modifier la valeur initiale (par défaut 0)

Exemple

```
for i in range(2, 5):  
    print(i)
```

Le résultat est

2
3
4



Exemple

Il est possible de modifier le pas (par défaut 1)

```
for i in range(0, 5, 2):  
    print(i)
```

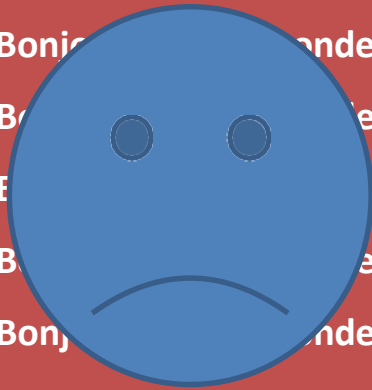
Le résultat est

0
2
4

Exemple : Afficher 10 fois "Bonjour tout le monde"



```
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )  
print ("Bonjour tout le monde" )
```



```
for i in range(10):
```

```
    print (" Bonjour tout le monde" )
```

Pour chaque itération de la boucle on affiche :

Bonjour tout le monde



Exemple : Ecrire un programme **Nombres.py** qui affiche les nombres entiers entre 0 et 20.

```
for i in range(21):  
    print(i)
```

Exemple : Ecrire un programme **Nombres-pairs.py** qui affiche les nombres pairs entre 0 et 20?.



Exemple

1. Calculez de la somme des nombres de 1 à 10
2. Calculez de la somme des nombres de 1 à n (avec n saisi par l'utilisateur)
3. Affichez des nombres de 10 à 1 en ordre décroissant :



Boucle while

while : à chaque itération on teste si la condition est vraie avant d'accéder aux traitements

```
while condition[s] :  
    # instruction[s]
```

la condition (dite condition de **contrôle de la boucle**) est évaluée à chaque itération. Les instructions (corps de la boucle) sont exécutés tant que la condition est vraie, on sort de la boucle dès que la condition devient fausse

Attention aux boucles infinies, vérifier que la condition d'arrêt sera bien atteinte après un certain nombre d'itérations

Pour manipuler des valeurs (de 0 a 4 **par exemple**) on peut utiliser aussi la Boucle while.



Exemple

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Le résultat est

0
1
2
3
4



Pour tout afficher sur une seule ligne

```
i = 0
while i < 5:
    print(i, end=" ")
    i += 1
```

Le résultat est

0 1 2 3 4



Exemple

Écrire un programme qui demande un entier positif, et le rejette tant que le nombre saisi n'est pas conforme.

```
N= float(input("Entrez un nombre positif"))  
while (N < 0):  
    N=float(input("Entrez un nombre positif"))
```



break

Exemple avec break

```
i = 0
while i < 5:
    print(i)
    if i == 2:
        break
    i += 1
```

Le résultat est

0
1
2



continue

Exemple avec continue

```
i = 0
while i < 5:
    i += 1
    if i == 2:
        continue
    print(i)
```

Le résultat est

```
1
3
4
5
```

Exemples



1. Affichez les diviseurs de n (avec n saisi par l'utilisateur)

2. Affichez le nombre de diviseurs de n (avec n saisi par l'utilisateur)

3. Calculez la somme de 10 nombres donnés par l'utilisateur

4. Calculez le max de 10 nombres donnés par l'utilisateur

5. Écrivez un programme qui permet à l'utilisateur de saisir une suite de nombres (entiers). Le programme doit calculer et afficher la moyenne de ces nombres. La saisie se termine lorsque l'utilisateur entre 0.



BOUCLE IMBRIQUEE

Boucle imbriquée signifie une instruction de boucle à l'intérieur d'une autre instruction de boucle.

Exemple

```
for i in range(5):  
    for j in range(4):  
        print(i,j)
```

0 0
0 1
0 2
0 3
1 0
1 1
1 2
1 3
2 0
2 1
2 2
2 3
3 0
3 1
3 2
3 3
4 0
4 1
4 2
4 3

Example :

Ecrire un programme permettant d'imprimer le triangle suivant:

```
1  
1 2  
1 2 3  
1 2 3 4  
1 2 3 4 5
```



Exemple

1. Écrivez un programme qui demande à l'utilisateur de saisir un nombre entier n . Ensuite, le programme doit vérifier si n est un nombre premier ou non.
2. Écrivez un programme qui demande à l'utilisateur de saisir un nombre entier n . Ensuite, le programme doit afficher tous les nombres premiers $\leq n$.